

AppLooper: An Agentic Application Engineering Loop for Accountable Release with Virtual-User Feedback

Zihong He
zhe154@connect.hkust-gz.edu.cn
HKUST (GZ)
Guangzhou, China

Chen Liang
chenliang2@hkust-gz.edu.cn
HKUST (GZ)
Guangzhou, China

Hai-Ning Liang
hainingliang@hkust-gz.edu.cn
HKUST (GZ)
Guangzhou, China

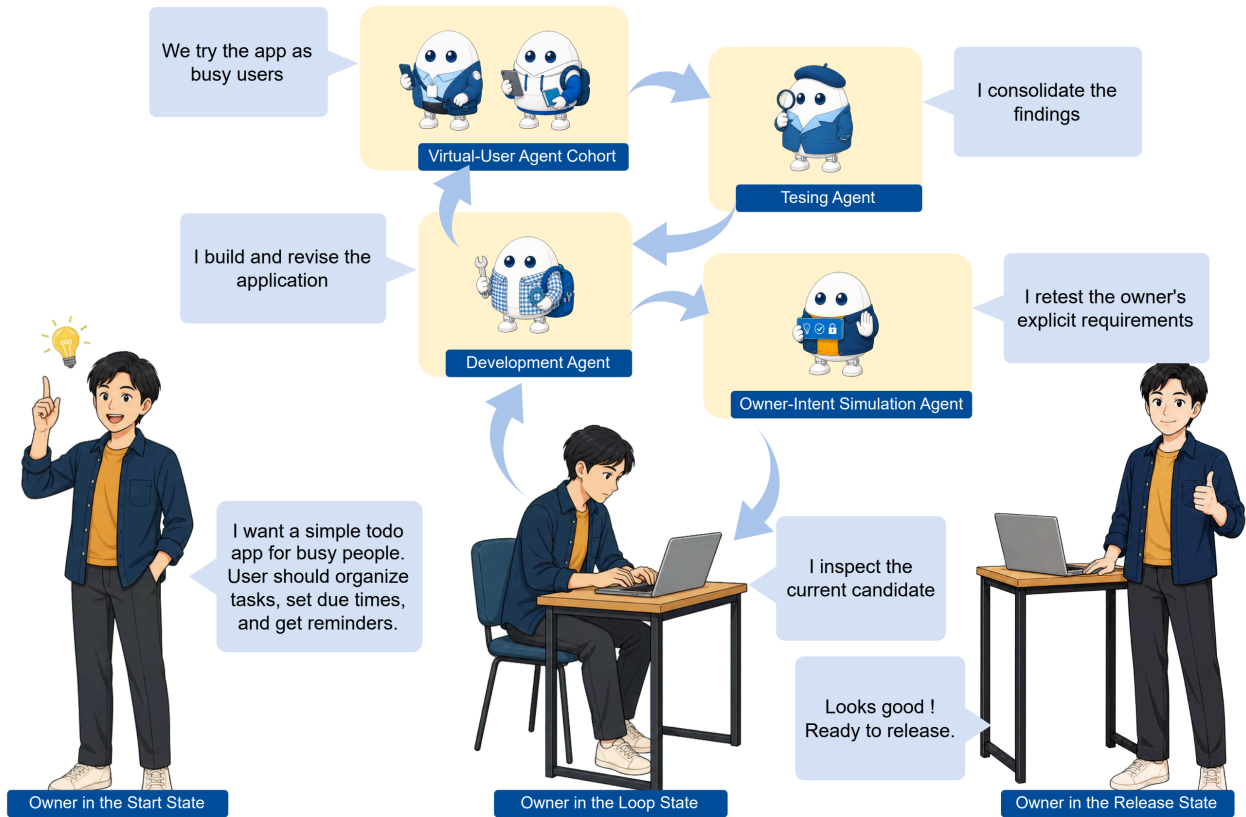


Figure 1: AppLooper connects an application owner with a development agent, a virtual-user agent cohort, a testing agent, and an owner-intent simulation agent from initial idea through iterative inspection and accountable release. The character illustrations were generated from author-written prompts using OpenAI’s image-generation capability; the workflow structure, labels, and composition were created by the authors.

Abstract

Much existing research on coding agents organizes application development as an iterative loop of requirement interpretation, implementation, tool execution, evaluation, and repair. As these loops run longer, requirements may drift; users may lose awareness of the current state and rationale for changes; and generated applications may remain insufficiently grounded in target users’ contexts and needs. Application engineering therefore requires a

mechanism connecting owner intent, target-user experience, development changes, and responsibility for release. We present AppLooper, a human-coding-agent-virtual-user application engineering loop for accountable release. An application owner confirms frozen requirements, supplies feedback, inspects candidates, and retains final release authority. A development agent produces and revises versioned candidates. A virtual-user agent cohort executes interface scenarios grounded in target users and contexts of use. Besides, an owner-intent simulation agent retests only requirements, constraints, and feedback explicitly confirmed by the owner, abstaining when evidence is insufficient. A testing agent performs

read-only developmental checks by reproducing reported failures, running existing regression tests, and exercising the current candidate through its browser interface. The orchestration layer groups the resulting findings and routes them into development revision, targeted retesting, and owner inspection. AppLooper binds requirements, feedback sources, interface targets, development changes, retesting outcomes, owner interactions, and release decisions to specific versions. It thereby extends sustained coding-agent iteration into a traceable and reviewable lifecycle in which humans retain final responsibility for release. Source code is available at <https://github.com/ZihongHe/applooper>.

Keywords

LLM-based coding agents, multi-agent systems, human-in-the-loop, human-agent interaction, virtual users, accountable software release

1 Introduction

Recent advances in large language models have substantially improved their capabilities in code generation, reasoning, and tool use, enabling them to serve as the core reasoning and decision-making components of coding agents. Coding agents connect these models to repositories, command-line interfaces, execution environments, browsers, and testing tools, allowing them to inspect software states, modify artifacts, and continue acting based on environmental feedback. Liu et al. [18] characterizes such systems as extending standalone language models through perception and access to external resources, while Yang et al. [34] and Wang et al. [31] demonstrate coding agents that navigate repositories, edit files, invoke commands, and run tests in executable environments.

Building on these capabilities, coding agents are expanding from local code generation to multi-step software engineering tasks spanning multiple files and execution stages. Jimenez et al. [16] shows that real-world software issues often require systems to coordinate information across functions, classes, and files while interacting continuously with an execution environment. Prior work has organized model actions, program execution, test feedback, and code repair into feedback-driven iterative processes. [35] interleaves reasoning, action, and environmental observation; [8] uses execution outcomes to support iterative debugging; and [15] improves generated programs through repeated test generation and execution. An architectural analysis of Claude Code, Liu et al. [19], characterizes its core as a simple while-loop that repeatedly invokes the model, executes tools, and continues until the interaction terminates; permission control, context compaction, extension mechanisms, sub-agent coordination, and session storage are implemented around this loop. Across these systems, iterative execution and feedback allow coding agents to revise failures incrementally and continue advancing toward predefined software tasks.

Such sustained automation also changes how users participate in development. Users can delegate more code editing, command execution, and local debugging to agents while shifting their attention toward goal specification, direction calibration, exception correction, and result verification. In a deployment study, engineers perceived that the collaboration in [27] could reduce the time and effort required for some planning and coding tasks, although they

remained concerned about code quality. As iterative development continues, three interrelated user-facing problems emerge: locally feedback-driven changes may drift from the application owner’s overall intent; users who leave a long-running development process may struggle to recover the current version, the rationale for changes, and the remaining risks; and passing tests or receiving a successful evaluator verdict may be insufficient to establish that an application supports its target users’ actual tasks. Tang et al. [28] identifies visible misalignment in project understanding, intent interpretation, rule compliance, operational scope, code execution, and progress reporting. Among visible cases that were eventually resolved, 91.49% still required explicit user correction. Developers’ calibrated trust also depends on establishing appropriate expectations, validating generated suggestions, and receiving interface information that supports evaluation, as reported in [30].

To address these problems, we present **AppLooper**, an application engineering loop for accountable release involving an application owner, a development agent, a virtual-user agent cohort, an owner-intent simulation agent, and a testing agent. The application owner first confirms the target users, representative contexts, core tasks, and requirement constraints, and subsequently provides feedback on specific requirements or the current application at designated development checkpoints. The development agent continuously implements and revises the application based on the frozen requirements and confirmed feedback. The virtual-user agent cohort consists of multiple simulated target users with documented persona provenance, applicable contexts, and uncertainty boundaries; these agents execute different browser-based task scenarios and surface potential issues. The owner-intent simulation agent conducts bounded, read-only retesting of the current candidate using only requirements, constraints, and feedback explicitly confirmed by the owner. The testing agent remains separate from implementation and performs read-only developmental checks that combine failure reproduction, existing regression tests, source-assisted diagnosis, and browser interaction with the current candidate. The orchestration layer groups the resulting findings for subsequent revision and retesting. These roles have distinct permissions and evidence boundaries, while the final release decision always remains with the application owner.

Figure 1 introduces this lifecycle, showing how the owner moves from an initial idea into the multi-agent feedback loop and ultimately inspects the current candidate before authorizing release.

AppLooper organizes these roles into a continuous **feedback – revision – retesting – acceptance** process. Owner feedback is first linked to stable requirement items and interface targets and is then applied by the development agent to a subsequent candidate. The testing agent performs regression and browser-based checks of that candidate; the virtual-user agent cohort executes both core scenarios and feedback-specific scenarios; and the owner-intent simulation agent retests the owner’s explicit feedback. The orchestration layer groups the resulting findings and routes them to the development agent. The development agent revises the application accordingly, after which the updated candidate undergoes layered testing and browser-based scenario verification. Owner

feedback, virtual-user findings, testing-agent findings, implementation changes, and retesting outcomes therefore remain associated with specific candidate versions. The application owner subsequently evaluates these intermediate development states. AppLooper converts each feedback cluster into a candidate-bound inspection scenario that presents the task in which the issue occurred, the associated interface evidence, the change introduced in the current version, and the target that now requires inspection. The owner enters an isolated executable interface, performs the relevant interactions, and returns a pass or return verdict for the scenario. A returned scenario re-enters the development loop, whereas a passed scenario becomes evidence that the corresponding issue has been inspected for that candidate. Each scenario records state preparation, actual interaction, the owner’s verdict, and temporary-data cleanup. When the candidate changes, scenario conclusions associated with the previous candidate are invalidated.

These intermediate feedback and acceptance records are then incorporated into the release context. For each immutable candidate, the system aggregates the frozen requirements, shared tests, critical user journeys, resolved and unresolved feedback clusters, scenario-retesting outcomes, owner inspection actions, and known limitations. During final acceptance, the owner first reviews the current candidate and its evidence boundaries, then completes key tasks in the executable application and makes a release decision for the version actually experienced. Any subsequent candidate change triggers renewed acceptance. Development feedback therefore accumulates through a traceable sequence of *feedback submission*, *implementation*, *independent retesting*, *owner acceptance of intermediate results*, and *final version inspection*, forming version-bounded release evidence. Results produced by the virtual-user agents and owner-intent simulation agent are explicitly labeled as simulated evidence and stored separately from real-participant data. Building on existing feedback-driven coding-agent architectures, AppLooper establishes an application engineering lifecycle that connects owner intent, implementation by the development agent, virtual-user experience, read-only developmental testing, orchestration-layer finding grouping, owner-intent retesting, and human release approval. Users participate in sustained development at low-frequency but consequential checkpoints and use feedback and evidence bound to specific candidates to understand, inspect, and approve the final artifact.

This work makes three contributions:

- (1) We introduce and implement AppLooper, which organizes the application owner, development agent, virtual-user agent cohort, owner-intent simulation agent, and testing agent into an end-to-end application engineering loop for accountable release.
- (2) We introduce an evidence-bounded development-feedback mechanism that associates owner feedback, virtual-user and testing-agent findings, and owner-intent retesting outcomes with stable requirements, interface targets, and candidate versions, and forms a continuous loop through implementation, orchestration-layer grouping, scenario retesting, and owner acceptance of intermediate results.
- (3) We introduce a version-bounded acceptance and evidence contract that associates intermediate feedback outcomes,

shared layered tests, critical user journeys, executable experience, and final approval with an immutable candidate, thereby connecting continuous development to human-authorized release.

2 Related Work

2.1 LLM-Driven Coding Agents and Feedback-Driven Loops

LLM-driven coding agents have changed how developers interact with software development interfaces. With code-completion tools, developers typically continue to operate the editor directly and individually read, accept, or revise local suggestions generated by the model. Mozannar et al. [23] identifies additional interaction and time costs associated with prompting, inspecting, editing, and validating code recommendations. Coding agents elevate this interaction to the task level: developers can describe a problem to be solved, while the agent navigates files, edits code, invokes commands, and executes tests through interfaces to the repository and execution environment. Yang et al. [34] treats language-model agents as a new class of computer users requiring specialized software interfaces, while Wang et al. [31] provides a general development platform combining a sandbox, command line, browser, and support for agent coordination. Developers’ interactions consequently shift in part from directly performing each coding operation to specifying tasks, observing agent actions, and inspecting generated artifacts.

Empirical studies further show that programmers use code-generating systems in different modes. Barke et al. [3] distinguishes an acceleration mode, in which programmers know the next step, from an exploration mode, in which they use generated suggestions to examine possible directions. Users may also prefer code-generation tools even when task-completion improvements are limited, while continuing to experience difficulty understanding, editing, and debugging generated code [29].

LLM-driven coding agents can now perform multi-step software engineering tasks in real-world repositories. The issues collected in [16] require models to edit real codebases in response to issue descriptions and commonly involve information distributed across multiple software components. Yang et al. [34] supports repository navigation, file editing, and test execution through a purpose-built agent-computer interface, while Wang et al. [31] extends these capabilities to general software-development and web-interaction tasks.

Some work coordinates complex tasks through interleaved reasoning and acting. Yao et al. [35] interleaves reasoning traces, actions, and environmental observations, enabling models to update subsequent actions in response to newly observed information. Such approaches help agents organize multi-step behavior, while code execution, test outcomes, and environmental states provide observable evidence for further adjustment.

Feedback-driven iteration further allows agents to reuse information from failed attempts. Madaan et al. [22] repeatedly uses the same model to generate feedback and revise its output, while Shinn et al. [26] converts task feedback into verbal reflections stored in episodic memory to shape subsequent attempts. In programming tasks, Chen et al. [8] combines code execution with natural-language explanations to support iterative debugging, whereas

Huang et al. [15] improves generated code through iterative testing and optimization involving specialized agents.

Beyond iterative refinement within a single model or a small generation–testing loop, prior work has organized software development through specialized agent roles. Qian et al. [24] coordinate agents across design, coding, and testing through structured multi-turn communication, while Hong et al. [14] encode standardized operating procedures into role-specific prompt sequences to structure multi-agent collaboration and intermediate verification. These systems demonstrate the value of role specialization in automated software production; AppLooper instead centers its role organization on target-user experience, read-only developmental testing, orchestration-layer finding grouping, owner-intent retesting, and human-authorized release.

These feedback-driven patterns also motivate practical long-running development workflows that place a coding agent within an external evaluation cycle. In our study, we instantiate this structure as the **Claude Code Loop**. Claude Code serves as the development agent, reading requirements, editing the workspace, and executing tests. Liu et al. [19] describes its model–tool runtime and supporting mechanisms for permissions, context management, extensibility, and session persistence. The outer bounded loop follows the evaluator–optimizer pattern described in Anthropic’s [2]: an independent evaluator checks each candidate against frozen requirements, shared critical user journeys, and layered tests, then returns feedback to Claude Code. Iteration stops when the candidate reaches the common acceptance threshold or the 50-step limit.

The resulting baseline combines Claude Code with an external *evaluation–feedback–redevelopment cycle*, using structured state and version records to preserve context across iterations. AppLooper builds on the same development agent and acceptance criteria, while additionally connecting owner feedback, virtual-user findings, owner-intent retesting outcomes, implementation changes, intermediate owner acceptance, and final release evidence to specific candidate versions.

2.2 Human Participation and Verification in Agentic Development

Traditional user-centered design emphasizes that user participation should extend throughout system development. Gould and Lewis [12] proposes early and continual focus on users, empirical measurement through actual use, and iterative design based on evaluation results. Kujala [17] similarly finds that user involvement can support requirements capture and user satisfaction, while participant roles, tacit-requirement elicitation, and practical costs require careful consideration. Prior work distinguishes *user participation*, referring to users’ concrete activities during system development, from *user involvement*, referring to the perceived importance and personal relevance of the system [4, 5]. Accordingly, message counts, clicks, or final satisfaction alone do not provide sufficient evidence of meaningful participation.

Coding agents shift part of human work from directly producing code toward goal setting, oversight, and verification. Takerngsak-siri et al. [27] introduces Human-in-the-loop LLM-based Agents (HULA), a framework that allows engineers to refine coding plans,

review generated code, and guide agents at multiple stages. In its deployment, engineers perceived that HULA could reduce planning and coding effort for some straightforward tasks while continuing to raise code-quality concerns. Yan et al. [33] supports code inspection by presenting lightweight explanations adjacent to generated code, while Mozannar et al. [23] demonstrates that reading, judging, and validating AI-generated code introduce additional interaction and time costs.

Direct causal evidence that developers trust code less simply because they did not personally monitor its generation remains limited. Existing research nevertheless shows that understanding and verification are important conditions for calibrated trust. Through interviews with 17 developers, Wang et al. [30] identifies establishing appropriate expectations, configuring AI tools, and validating AI suggestions as central challenges in trust formation. Systems therefore need to provide sufficient provenance, revision history, and verification evidence when users re-enter a process so that they can form calibrated judgments.

More broadly, established human–AI interaction guidelines emphasize communicating system capabilities, supporting correction when the system is wrong, and providing information that helps users understand and control AI behavior [1]. These principles motivate presenting candidate state, evidence boundaries, known limitations, and available owner actions when users re-enter a long-running agentic process.

HULA demonstrates that human feedback can enter the planning and code-generation stages, with professional developers primarily reviewing coding plans and source code. Dhanorkar et al. [10] further identifies four forms of oversight work—a priori control, co-planning, real-time monitoring, and post-hoc review—showing that oversight includes preventive, collaborative, concurrent, and retrospective activities.

Agents may still interpret goals and rules differently from users’ intentions. Tang et al. [28] observes misalignment involving project understanding, intent interpretation, rule compliance, operational scope, code implementation and execution, and progress reporting in real-world sessions. The study operationalizes misalignment through breakdowns made visible by developer pushback; among visible resolutions, 91.49% required explicit user correction. This observation suggests that long-running coding-agent collaboration requires persistent connections among user goals, feedback, and the current application version.

Building on these forms of human participation and oversight, AppLooper organizes user involvement as an evidence loop spanning development and release. The owner confirms target users and tasks at the outset and provides feedback on specific requirements or current candidates during development. The development agent processes this feedback; the owner-intent simulation agent performs bounded retesting based on explicit owner input; the virtual-user agent cohort contributes scenario findings from documented target-user perspectives; and the testing agent performs read-only regression and browser-based checks. The orchestration layer groups the resulting findings for subsequent revision and retesting. The owner then accepts intermediate outcomes through actual interface use, after which accepted feedback and unresolved risks enter the final release context.

2.3 Software Traceability and Version-Bounded Evidence

Requirements traceability has long distinguished links to the origins and stakeholder context of requirements from links among requirements and subsequent development artifacts [11]. Broader software-traceability research emphasizes creating, maintaining, and using trustworthy links among evolving software artifacts, while noting that ad hoc, after-the-fact tracing often fails to realize its intended benefits [9]. AppLooper brings this concern into sustained agentic application development by associating frozen requirements, owner feedback, virtual-user and testing-agent findings, orchestration-layer feedback groups, owner-intent retesting outcomes, implementation changes, and approval with explicit candidate versions.

2.4 Virtual Users and Formative UX Evaluation

Because real-user testing may not fully match the pace of rapid prototype iteration, some work uses LLMs to simulate interactions between particular users and applications and to provide early formative signals. Xiang et al. [32] simulates mobile-application interactions based on user characteristics, contexts of use, and tasks, and generates heuristic usability feedback. In its evaluation of a simple smartwatch interface, the similarity between issues identified by SimUser and by real users varied from 35.7

Lu et al. [21] combines a persona generator, an LLM-agent module, a universal browser connector, and a result-viewing interface to generate and review simulated website interactions. Its evaluation with 16 UX researchers found that participants recognized the system’s exploratory value while also raising concerns about the future use of LLM agents in UX studies.

Executing long-horizon tasks in realistic web applications remains difficult even apart from persona simulation. WebArena evaluates agents on reproducible, fully functional websites and reports a substantial gap between its strongest GPT-4-based baseline and human task success [36]. AppLooper therefore treats successful browser execution as task-specific evidence with explicit limitations rather than as proof that a virtual user reliably represents human experience.

Initializing virtual users also requires representing variation within a target population. Bui et al. [6] combines weighted personas and exemplars to improve distributional alignment and diversity in its synthetic-data tasks. Liu et al. [20] further argues that individually plausible conversations can still distort the overall composition of behaviors and models population-level distributions through prevalence-weighted behavioral groups. These studies support explicitly documenting the composition, weights, and provenance of virtual-user cohorts, while limiting claims of population representativeness to settings with defensible reference distributions.

A gap nevertheless remains between virtual and real users. Seshadri et al. [25] finds that the choice of user LLM can affect estimates of agent performance, that simulated users can be differently calibrated across task difficulty, and that proxy effectiveness can vary across languages and populations. Although this work examines conversational users in agent benchmarks rather than

application-interface usability, it shows that virtual-user behavior and evaluation must be interpreted in relation to the underlying model, target population, and task conditions. Together with SimUser’s varying coverage across user groups and usability categories and UX researchers’ concerns about UXAgent, these findings position virtual users as bounded sources of formative experience evidence.

Building on these virtual-user and formative-evaluation methods, AppLooper places a virtual-user agent cohort within an ongoing application-development process and constrains its role through complementary agent responsibilities. The virtual-user agent cohort executes scenarios using target-user personas with documented provenance and applicability boundaries. The testing agent performs read-only regression and browser-based checks of the current candidate. The orchestration layer groups the resulting findings for subsequent revision and retesting. The owner-intent simulation agent retests only requirements and feedback explicitly expressed by the owner. The development agent revises the application using these inputs. All simulated results remain linked to candidate versions and interface targets and are labeled as simulated. The application owner accepts relevant scenarios through actual interaction and retains responsibility for the final release decision.

3 System Overview and Implementation

AppLooper is a human-coding-agent-virtual-user application engineering system for sustained development and accountable release. The screenshots of AppLooper are shown in Figure 2. An application owner first specifies the target users, application type, and core problem to be solved. After AppLooper determines that the request can enter construction, it starts continuous development, simulated experience, internal testing, and candidate acceptance around that frozen requirement. The lifecycle is organized around versioned application candidates so that owner feedback, simulated findings, development revisions, retesting outcomes, and release decisions remain continuously associated with a specific candidate.

To realize this lifecycle, an application-management orchestration layer coordinates five actors: the application owner confirms requirements and makes the final release decision; the development agent continuously implements and revises the application in an isolated workspace; the virtual-user agent cohort executes interface scenarios grounded in target users and contexts of use; the owner-intent simulation agent performs read-only retesting based only on requirements, constraints, and feedback that the owner has already confirmed; and the testing agent performs read-only developmental checks through failure reproduction, existing regression tests, source-assisted diagnosis, and browser interaction. The orchestration layer groups the resulting findings for subsequent revision and retesting. The construction and use of virtual-user personas draw on [6] and [20]. The organization of interactive usability simulation can be compared with [32] and [21].

To make these role boundaries visually legible, the AppLooper interface uses distinct character illustrations generated from author-written prompts with OpenAI’s image-generation capability. These illustrations are presentational assets only and do not affect agent prompts, permissions, inference, evidence processing, or release decisions.

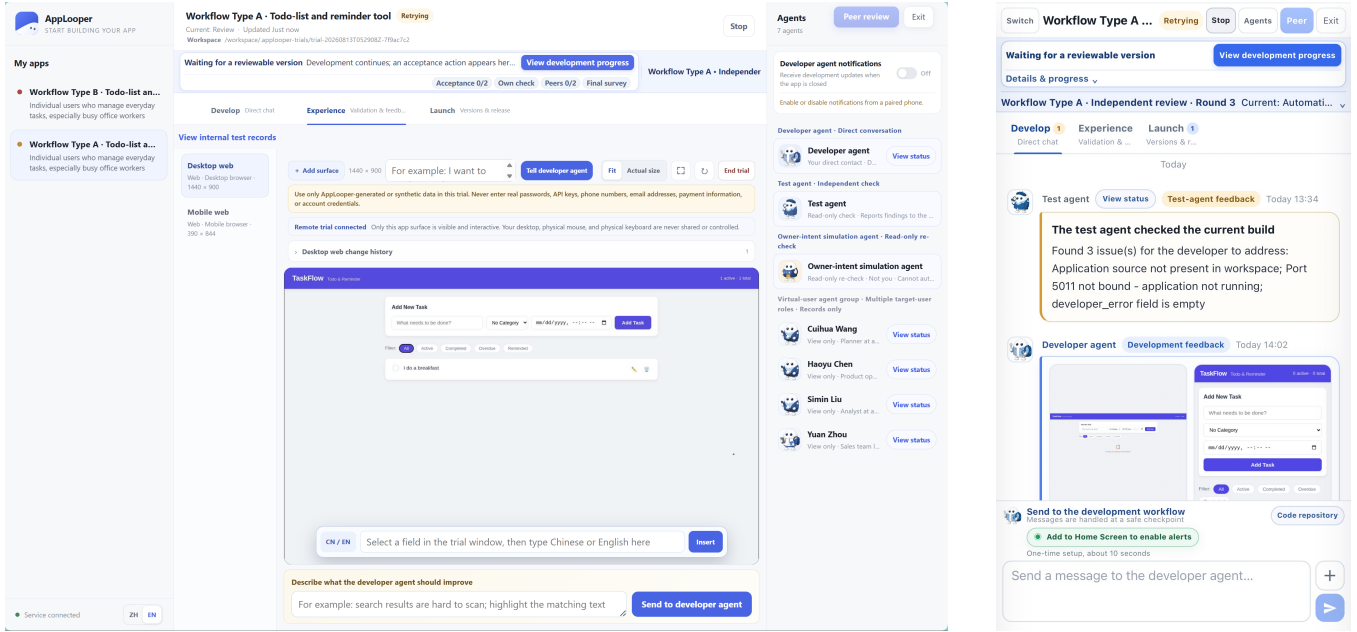


Figure 2: Snapshots of AppLooper opened on a personal computer (left) and a mobile phone (right). The personal-computer interface presents an experience window for the developed application, allowing owner to experience in real time the application built by the development agent. The mobile interface presents loop-based feedback and interaction involving multiple agents.

In terms of model and runtime configuration, AppLooper uses Claude Code as the coding runtime of the development agent. Online application-feasibility judgment and other auxiliary steps that require independent structured decisions use Agnes 2.5-flash with sampling temperature fixed at 0 to reduce judgment variance. The roles do not share one free-form conversation. Instead, the orchestration layer injects role prompts, the frozen requirement, the current candidate, and structured-output constraints at each stage. Prompt templates are maintained in both Chinese and English and are selected according to the interface language, so sustained construction is organized as an auditable state machine rather than one-shot code generation.

AppLooper therefore forms the following continuous lifecycle: **frozen requirement** → **development** → **developmental testing** → **simulated experience** → **finding grouping** → **development revision** → **targeted retesting** → **owner inspection** → **candidate release**.

Each piece of feedback remains continuously linked to its source, interface target, revision outcome, retesting evidence, owner inspection, and corresponding candidate version.

3.1 AppLooper Application Feasibility Judgment

Before development starts, AppLooper judges whether the requested application falls within a scope that can be prototype-validated safely and comparatively. The purpose of this mechanism is admission control over research and deployment boundaries; it does not predict the eventual construction success rate. Judgment proceeds in two stages. The second stage acts only on requests that the first

stage has not rejected. Requests already rejected in the first stage do not enter model judgment and therefore cannot be overturned to an admit decision.

The first stage is a deterministic scope policy. The orchestration layer normalizes and concatenates the owner’s target-user, application-type, and problem descriptions, then matches them against a fixed keyword lexicon under case-insensitive comparison, in a fixed order. Requests that match credential theft, malware, or circumvention of security controls are judged non-constructible. Requests that match serious domains such as medical diagnosis or treatment, autonomous driving or real-world vehicle control, weapons and nuclear facilities, or high-risk financial or legal decision-making are likewise judged non-constructible, with the matched category reported. Requests that match clearly out-of-scope high-difficulty goals such as artificial general intelligence or training a foundation model from scratch are also rejected. If a request depends on real sensitive accounts, production systems, or hardware control that is unavailable in the current environment, but can be rewritten as a safe prototype that uses only fictional sample data and simulated interfaces, the stage returns a rewritten frozen requirement for owner confirmation. Otherwise the request is judged admissible. For example, a diagnostic assistant aimed at hospital patients is rejected in the first stage, whereas a personal todo-and-reminder tool may proceed to the second stage.

The second stage is online model refinement. For requests not rejected by the first stage, the orchestration layer selects the locale-matched prompt template according to the current interface language and requires a single JSON object as the response. The English prompt template is:

You are a strict admission gate for a software-prototype user study. Output only one JSON object. The decision field must be one of PASS, REPHRASE, or BLOCK. Ordinary and safe software ideas should PASS. Only low-difficulty and non-serious applications are allowed. Medical diagnosis/treatment, autonomous driving or vehicle control, weapons and nuclear facilities, high-risk financial or legal decision-making, and clearly high-difficulty goals such as AGI or training a foundation model from scratch must be BLOCK. If the request depends on unavailable hardware, real sensitive accounts, or production systems but a safe simulated prototype is feasible, use REPHRASE. If it is clearly malicious, illegal, credential-stealing, or cannot be safely rewritten, use BLOCK. The gate is not a success-probability predictor. For REPHRASE, return frozen_requirement with target_user, app_type, and problem; for PASS, return frozen_requirement unchanged; also provide a short reason. Requirement: {"target_user":"...", "app_type":"...", "problem":"..."}

The model returns admit, rewrite, or reject, together with a complete three-field frozen requirement. If the model is unavailable, times out, or returns an invalid structure, the orchestration layer falls back to the first-stage deterministic result. After the owner confirms the final frozen requirement, the orchestration layer compiles it into a development intent. For the todo example above, the intent takes the form “develop a todo-list and reminder tool for individual users who manage everyday tasks; support recording and categorizing todos, setting due times, and reminding before a task is due.” The same data-and-AI capability boundary is attached for both workflows: owner-uploaded materials, provenance-bearing retrieval snapshots, explicitly labeled synthetic test data, and owner-configured inference services are allowed; secret values are not written into source code.

3.2 Roles and Responsibility Boundaries

AppLooper distinguishes the application owner, development agent, virtual-user agent cohort, owner-intent simulation agent, and testing agent, and coordinates their interaction through the application-management orchestration layer.

Application Owner. The application owner confirms the frozen requirement, supplies feedback during development, and, after a candidate reaches acceptance, completes inspection and final release through the actual interface.

Development Agent. The development agent continuously implements and revises the application. In the current implementation, Claude Code serves as the coding runtime. In each turn, the orchestration layer provides the current task, frozen requirement, task dependencies, confirmed feedback, and previous-round state, rather than an unbounded full chat history. The development agent operates in an isolated workspace, performs code changes and tests, and returns the current candidate together with development evidence. Its English role-prompt template is:

You are the only developer agent that may modify source code in the current workspace. Inspect existing code first and build on it; do not reset, stash, clean, rebase, commit, or destroy the user's prior edits. Strictly obey active guardrails. Bugs, small in-guardrail features, and usability issues from experience feedback may be implemented directly; unapproved major capabilities or large changes to existing capabilities must be deferred. Ambiguous, conflicting, or low-confidence decisions must keep the current implementation or take the most conservative reversible default; questions and blockers must default to empty. Do not call interactive questioning tools. Do not contact users through external channels; return structured results only. Run relevant tests, prepare a real experience entry point, and preserve screenshot evidence when possible. The runnable experience must expose a fully started, openable web preview base address; specific page paths belong to view routes. Before submission, actually visit the entry and each declared route and confirm they load.

Virtual-User Agent Cohort. The virtual-user agent cohort consists of multiple simulated target users with explicit personas and contexts of use. Each agent completes end-to-end tasks on

the current candidate through real browser interaction and produces formative findings associated with concrete pages, controls, and interaction traces. The experience role prompt injected by the orchestration layer specifies that virtual users play the assigned persona, do not modify source code, complete tasks through real interface operations, do not substitute source reading or API calls for experience, submit only problems they actually encounter, and report failure when operation is impossible. When an application involves login or registration, virtual users use synthetic test identities and a controlled verification flow provided by the research environment; they do not use real personal identity information or trigger real communication.

For the todo application above, the orchestration agent generates personas that cover the declared target-user segments during planning. Each persona includes at least an identifier, name, covered user segments, demographic descriptors such as age and gender, location, role, technical proficiency, device, motivation, constraints, usage context, habits, and a task script composed of concrete action steps. For example, a persona of an office worker who needs to capture due items quickly in fragmented time may include the steps: open the application and find the create entry, create a categorized todo with a due time, and confirm that the todo appears in the list with a visible due time. When reliable evidence is missing, the orchestration agent stops making the corresponding demographic or interest inferences, and those inferences are not written as factual population claims.

Owner-Intent Simulation Agent. The owner-intent simulation agent corresponds to a single application owner. Its inputs are limited to the frozen requirement, confirmed constraints, and feedback that the owner has explicitly expressed. It performs at most two rounds of read-only retesting on the relevant candidate; when evidence is insufficient, it abstains and retains the case as unresolved risk. The owner remains responsible for actual interface inspection and formal release, and may skip this retesting at any time. Its English role-prompt template is:

You are a read-only product-owner intent agent, not a target-user persona and not the real owner. You may only use the frozen intent, explicit user messages, confirmed constraints, and their source identifiers provided item-by-item in the input. Use real computer use to inspect one repaired scenario. Do not infer demographics, personal preferences, or unstated needs; abstain when evidence is insufficient. Do not modify source code, guardrails, acceptance results, or release conclusions, and do not speak as the owner. Output is only simulated owner-intent evidence.

Testing Agent. The testing agent performs read-only gray-box developmental testing of the current runnable candidate. It first reproduces the development agent's latest reported or recovered failure, may inspect relevant source to diagnose the root cause, runs existing regression tests, and exercises the current task, critical interfaces, and visible pages through a real browser. It reports only observed defects and does not modify product source. The orchestration layer subsequently groups testing-agent and virtual-user findings for development revision and retesting; formal evaluation and final release remain the responsibilities of the evaluation mechanism and the application owner. Its English role-prompt template is:

You are AppLooper's read-only developmental test agent. Inspect the current runnable build and never modify product source. Reproduce the developer's latest reported or recovered failure first, then check the current task, critical interfaces, and visible UI. When the same command already failed, diagnose the root cause before running it again. Exercise runnable web UI

through a real browser and record evidence. Report only observed defects; do not treat later unimplemented tasks as defects. Do not ask questions, publish, or contact real users.

Application-Management Orchestration Layer. The application-management orchestration layer maintains feedback routing, task dependencies, candidate versions, testing state, and feedback-loop status, and schedules roles at safe development checkpoints. It constitutes the lifecycle state layer among roles and does not substitute for coding, simulated testing, or final release.

3.3 Continuous Feedback–Development–Retesting Loop

The core implementation of AppLooper is a sustained development loop organized around feedback and candidate versions.

After development begins, a read-only orchestration agent decomposes the frozen requirement into three stable structures: confirmed constraints, requirement tasks, and virtual-user personas. The orchestration agent does not modify source code; only content that the owner has explicitly stated or approved becomes a constraint; inferences are not elevated to constraints; and the planning stage does not ask the owner clarifying questions. Its planning prompt specifies a structured plan covering a summary, a constraint set, required user segments, coverage dimensions, a persona list, and a task list. Each constraint is represented as an object that includes at least constraint text, verifiable acceptance conditions, and a contiguous verbatim excerpt taken from the development intent without rewriting or translation. Each task includes an identifier, title, dependencies, associated constraints, initial status, and verification mode, thereby forming an executable task-dependency graph.

Taking the todo application as an example, the orchestration agent may extract three constraints from the intent: support recording and categorizing todos, with the source excerpt corresponding to “record and categorize todos”; support setting due times, with the source excerpt corresponding to “set due times”; and support reminding before a task is due, with the source excerpt corresponding to “remind before a task is due.” The corresponding tasks may be decomposed into “implement todo creation and categories,” “implement due times and reminder settings,” and “verify the create-and-inspect core path,” with the latter two depending sequentially on the former. The orchestration layer projects these tasks onto a requirement roadmap that shows only not started, in progress, implemented pending verification, verified, blocked, or deferred, together with corresponding evidence and next steps; it does not display subjective completion percentages or estimated times.

The development agent revises the application according to the current task and produces a versioned candidate. After the candidate enters the experience stage, the virtual-user agent cohort executes core or targeted scenarios. The orchestration layer also maintains a set of critical user journeys instantiated from the frozen requirement—for example, opening the application from a fresh session and finding the primary-task entry, completing the primary record–categorize–set-due-time task as the target user, and confirming that the result is clearly visible in the interface without blocking errors. The owner-intent simulation agent retests only feedback that the owner has already expressed explicitly.

Findings submitted by virtual users and the testing agent are grouped by the orchestration layer into feedback clusters, which retain the reporting identifiers, evidence summaries, priority, and applicable candidate. The orchestration layer then generates a versioned targeted scenario for each feedback cluster, and the scenario-management component records the applicable candidate, preconditioned state, interaction steps, expected result, actual result, owner verdict, temporary-state cleanup, and evidence provenance. Scenarios complete in the order open → interact → judge → clean up temporary state. Cases that only open a page without actual interaction, or that omit cleanup, are not counted as complete. When the candidate changes, conclusions from previous scenarios become invalid.

The development agent revises the application according to these structured feedback items and produces a new candidate. After revision, test maintenance and the orchestration layer reschedule four layers of testing: unit tests for pure logic, integration tests across components, fixture-replay tests for repeatable state, and real browser interaction tests. Targeted scenarios associated with specific feedback are re-executed by the scenario-management component at the same time. Feedback status is updated continuously from retesting outcomes, and the requirement roadmap advances the corresponding items in parallel.

This feedback loop continues the idea of feedback-driven subsequent attempts in [22] and [26]. AppLooper further connects feedback to concrete application versions, user perspectives, and subsequent owner inspection.

At key development points, the application owner may re-enter the process. The orchestration layer presents candidate-bound inspection information: who encountered what in which scenario, what evidence is currently available, what changed in this version, and what should be inspected. For example, for the finding “due time missing after creating a todo” reported by the virtual user “busy office worker,” the interface states that this round now shows due times in the list and detail views, and asks the owner to create one categorized todo with a due time in the executable preview to confirm. Presenting explanation next to the object to be inspected is consistent with the anchored-explanation approach in [33]. Related mixed-initiative web-automation work uses step-by-step natural-language descriptions, visual correspondence to interface elements, and step-through debugging to support understanding and validation of generated behavior [7]. AppLooper applies a related principle at the candidate level by pairing an issue, its evidence, the implemented change, and the interface target that the owner should inspect.

A piece of feedback in AppLooper therefore forms a complete lifecycle:

feedback → feedback cluster → development revision → candidate → retesting → owner inspection → candidate-bound evidence.

3.4 Candidate-Bound Acceptance and Release

Application release in AppLooper comprises two consecutive stages: a system-side candidate threshold and owner acceptance.

First, the current candidate enters candidate evaluation that is separated from the development session. An evaluation agent

makes a structured judgment from frozen constraints, layered-test results, critical-user-journey evidence, and severe-defect status. Its role prompt specifies independent candidate review: it reads product files and test evidence, and returns per-constraint outcomes, whether regression tests actually ran and passed, an issue list, screenshots, and completed-task identifiers. Issue severity is distinguished as low, medium, high, critical, and blocker. The orchestration layer treats blocker and critical issues as highest-priority defects, and high and medium issues as important defects that still require explicit handling. Shared tests are actually executed, and critical user journeys are completed in a real browser with evidence saved. After a candidate reaches the prescribed threshold, it is frozen as an immutable acceptance version and enters owner inspection. Autonomous iteration stops at the first candidate that meets the threshold; only a highest-priority defect that blocks a core task reopens the repair, regression, browser-retesting, and acceptance chain.

All testing, feedback, retesting, and inspection evidence is recorded against the corresponding candidate version by the orchestration layer and the scenario-management component. When later development produces a new version, that new candidate re-enters the corresponding testing and acceptance chain, so each judgment has an explicit version boundary. Approval in AppLooper is bound to a candidate: after the candidate changes, prior approval becomes invalid.

During owner acceptance, the orchestration layer summarizes the current candidate through a bounded checklist that includes the frozen requirement, critical user journeys, current test results, unresolved high-priority issues, viewed and pending feedback, simulated-evidence labels, and known limitations. At the same time, the interface highlights the most important candidate-bound feedback and presents a continuous “issue–evidence–change–inspection target” narrative; the full feedback list and technical details remain accessible but are collapsed by default. Because oversight interfaces can affect inspection behavior and confidence differently, AppLooper retains actual interface trial as the formal release step; related oversight research is discussed in [13]. Checklist completion or summary exposure is not coded as release correctness.

The owner then enters an isolated experience window, performs the core tasks, and decides to pass, return, or defer. The final release record continuously associates the frozen requirement, candidate version, evaluation result, evidence references, experience record, and final decision. Virtual-user testing and owner-intent retesting continue to be stored under a simulated-evidence label and are recorded separately from real owner actions and later operations data. In this way, final release in AppLooper can answer which candidate is being approved, which requirements and evidence that candidate corresponds to, which feedback items have already undergone revision and retesting, and on what basis the owner completed final inspection.

4 Conclusion

As coding-agent iterations increase, coding agents can continuously use execution outcomes, test feedback, and environmental states to advance application construction, turning application development into a sustained application engineering process. In this

process, application owner intent, target-user experience, development changes, and final release responsibility need to remain continuously connected. Existing research has separately explored coding-agent iteration, human participation, and virtual-user evaluation. Together, these directions leave a lifecycle-level question: how can “owner feedback, target-user experience evidence, implementation changes, and release decisions” remain associated with specific application candidates as an application continuously evolves? We present **AppLooper**, a human–coding-agent–virtual-user application engineering loop for accountable release. Through a continuous **feedback – revision – retesting – acceptance** process, AppLooper associates owner feedback, virtual-user findings, and read-only developmental-testing results with frozen requirements, interface targets, and versioned application candidates, and further connects them through orchestration-layer grouping, development revisions, owner-intent retesting, and owner inspection. Its **candidate-bound acceptance and release** mechanism binds layered tests, critical user journeys, feedback outcomes, actual interface experience, and final approval to the immutable candidate actually inspected by the owner. Through these mechanisms, AppLooper extends sustained coding-agent iteration into an application engineering lifecycle in which application intent, target-user experience, development changes, and human release responsibility remain traceably connected from initial application intent to human-authorized release.

References

- [1] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N Bennett, Kori Inkpen, et al. 2019. Guidelines for human-AI interaction. In *Proceedings of the 2019 chi conference on human factors in computing systems*. 1–13.
- [2] Anthropic. 2024. Building Effective Agents. <https://www.anthropic.com/engineering/building-effective-agents>
- [3] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.
- [4] Henri Barki and Jon Hartwick. 1989. Rethinking the concept of user involvement. *MIS quarterly* 13, 1 (1989), 53–63.
- [5] Henri Barki and Jon Hartwick. 1994. Measuring user participation, user involvement, and user attitude. *MIS quarterly* 18, 1 (1994), 59–82.
- [6] Ngoc Bui, Hieu Trung Nguyen, Shantanu Kumar, Julian Theodore, Weikang Qiu, Viet Anh Nguyen, and Rex Ying. 2025. Mixture-of-personas language models for population simulation. In *Findings of the Association for Computational Linguistics: ACL 2025*. 24761–24778.
- [7] Weihao Chen, Xiaoyu Liu, Jiacheng Zhang, Ian Long Lam, Zhicheng Huang, Rui Dong, Xinyu Wang, and Tianyi Zhang. 2023. Miwa: Mixed-initiative web automation for better user control and confidence. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 1–15.
- [8] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching large language models to self-debug. In *International Conference on Learning Representations*, Vol. 2024. 8746–8825.
- [9] Jane Cleland-Huang, Orlena CZ Gotel, Jane Huffman Hayes, Patrick Mäder, and Andrea Zisman. 2014. Software traceability: trends and future directions. In *Future of software engineering proceedings*. 55–69.
- [10] Shipi Dhanorkar, Samir Passi, and Mihaela Vorvoreanu. 2026. Human oversight of agentic systems in practice: Examining the oversight work, challenges, and heuristics of developers using software agents. In *The 2026 ACM Conference on Fairness, Accountability, and Transparency*. 6438–6465.
- [11] Orlena CZ Gotel and CW Finkelstein. 1994. An analysis of the requirements traceability problem. In *Proceedings of IEEE international conference on requirements engineering*. IEEE, 94–101.
- [12] John D Gould and Clayton Lewis. 1985. Designing for usability: key principles and what designers think. *Commun. ACM* 28, 3 (1985), 300–311.
- [13] Madeleine Grunde-McLaughlin, Hussein Moazzan, Maya Murad, Jingya Chen, Saleema Amershi, and Adam Fourney. 2026. Overseeing Agents Without Constant Oversight: Challenges and Opportunities. *arXiv preprint arXiv:2602.16844* (2026).
- [14] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Steven Yau, Zijuan Lin, Liyang Zhou, et al. 2024.

- MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, Vol. 2024. 23247–23275.
- [15] Dong Huang, Jie M Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010* (2023).
- [16] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues?. In *International Conference on Learning Representations*, Vol. 2024. 54107–54157.
- [17] Sari Kujala. 2003. User involvement: a review of the benefits and challenges. *Behaviour & information technology* 22, 1 (2003), 1–16.
- [18] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2024. Large language model-based agents for software engineering: A survey. *ACM Transactions on Software Engineering and Methodology* (2024).
- [19] Jiacheng Liu, Xiaohan Zhao, Xinyi Shang, and Zhiqiang Shen. 2026. Dive into Claude Code: The Design Space of Today’s and Future AI Agent Systems. *arXiv preprint arXiv:2604.14228* (2026).
- [20] Xinyi Liu, Rinat Khaziev, Hooshang Nayyeri, Emine Yilmaz, Charith Peris, and Hari Thadakamalla. 2026. Beyond Individual Personas: Aligning Synthetic Dialogue to Population-Level Behavior Distributions. *arXiv preprint arXiv:2606.07893* (2026).
- [21] Yuxuan Lu, Bingsheng Yao, Hansu Gu, Jing Huang, Jessie Wang, Yang Li, Jiri Gesi, Qi He, Toby Jia-Jun Li, and Dakuo Wang. 2025. Uxagent: A system for simulating usability testing of web design with llm agents. *arXiv preprint arXiv:2504.09407* (2025).
- [22] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems* 36 (2023), 46534–46594.
- [23] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2024. Reading between the lines: Modeling user behavior and costs in AI-assisted programming. In *Proceedings of the 2024 CHI conference on human factors in computing systems*. 1–16.
- [24] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. 2024. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*. 15174–15186.
- [25] Preethi Seshadri, Samuel Cahyawijaya, Ayomide Odumakinde, Sameer Singh, and Seraphina Goldfarb-Tarrant. 2026. Lost in simulation: LLM-simulated users are unreliable proxies for human users in agentic evaluations. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 47423–47439.
- [26] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems* 36 (2023), 8634–8652.
- [27] Wannita Takerngsaksiri, Jirat Pasuksmit, Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Ruixiong Zhang, Fan Jiang, Jing Li, Evan Cook, Kun Chen, and Ming Wu. 2025. Human-in-the-loop software development agents. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 342–352.
- [28] Ningzhi Tang, Chaoran Chen, Gelei Xu, Yiyu Shi, Yu Huang, Collin McMillan, Tao Dong, and Toby Jia-Jun Li. 2026. How coding agents fail their users: A large-scale analysis of developer-agent misalignment in 20,574 real-world sessions. *arXiv preprint arXiv:2605.29442* (2026).
- [29] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts*. 1–7.
- [30] Ruotong Wang, Ruijia Cheng, Denae Ford, and Thomas Zimmermann. 2024. Investigating and designing for trust in ai-powered code generation tools. In *Proceedings of the 2024 ACM conference on fairness, accountability, and transparency*. 1475–1493.
- [31] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2025. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, Vol. 2025. 65882–65919.
- [32] Wei Xiang, Hanfei Zhu, Suqi Lou, Xinli Chen, Zhenghua Pan, Yuping Jin, Shi Chen, and Lingyun Sun. 2024. Simuser: Generating usability feedback by simulating various users interacting with mobile applications. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–17.
- [33] Litao Yan, Alyssa Hwang, Zhiyuan Wu, and Andrew Head. 2024. Ivie: Lightweight anchored explanations of just-generated code. In *Proceedings of the 2024 CHI conference on human factors in computing systems*. 1–15.
- [34] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.
- [35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).
- [36] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. 2024. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, Vol. 2024. 15585–15606.